Digital Logic

GATE CSE 2027 · complete formula, rule, method & trap sheet

90 reference boxes · **11** chapters ·

Number Systems & Computer Arithmetic | Boolean Algebra & Logic Gates | Minimisation: K-Maps & Quine–McCluskey | Combinational Circuits | Latches & Flip-Flops | Counters & Shift Registers | Finite State Machines & Synchronous Design | Memory & Programmable Logic | Arithmetic Circuits & Error-Control Codes | High-Yield Sequential & Boolean Methods | Last-minute one-liners

1 Number Systems & Computer Arithmetic

Positional systems & conversion

NOTES

A number in base *r* is a sum of digit × $r^{position}$. Common bases: binary (2), octal (8, groups of **3 bits**), hexadecimal (16, groups of **4 bits**). To convert integer → base r, divide repeatedly by r and read remainders **bottom-up**; for the fractional part multiply repeatedly by r and read the integer parts **top-down**.

WORKED EXAMPLE

Worked numerical — conversions

```
(1101.101)₂ = 8 + 4 + 0 + 1 + 1/2 + 0 + 1/8 = 13.625
156 ÷ 2 …  = (10011100)₂ = (234)₈ = (9C)₁₆        [group bits: 010 011 100 → 2 3 4 ; 1001 1100 → 9 C]
(0.1)₁₀ in binary = 0.000110011001…  (non-terminating — why 0.1 is inexact in floating point)
```

Digits needed for the unsigned value N in base r: $\lceil \log_r(N+1) \rceil$. n bits hold 0 … 2^n–1.

Signed number representations (n bits)

REFERENCE TABLE

Scheme	How negatives are formed	Range	Zeros
Sign-magnitude	MSB = sign, rest = |x|	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	2 (+0, −0)
1's complement	invert every bit	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	2
2's complement	invert every bit, add 1 (= $2^n - x$)	**-2^{n-1} … $+(2^{n-1}-1)$**	1

FORMULA / RULE

Worked numerical — representing −45 in 8 bits

```
+45 = 0010 1101
1's complement of 45 : 1101 0010   (= −45 in 1's complement)
2's complement of 45 : 1101 0011   (= −45 in 2's complement)
Check: 256 − 45 = 211 = 1101 0011 ✔
```

8-bit 2's complement range: **−128 … +127**. Subtraction is done as addition: `A − B = A + (2's complement of B)`.

```
13 − 6 in 5 bits:   6 = 00110 → 1's complement 11001 → 2's complement 11010
  01101 + 11010 = 1 00111 → discard the carry-out → 00111 = 7 ✔
```

GATE FAVOURITE

GATE angle

2's complement has the **asymmetric range** (one more negative than positive), and sign extension (copy the MSB leftwards) preserves the value. Only 2's complement has a **unique zero**.

WORKED EXAMPLE

Worked numerical — 8-bit overflow test

```
  0111 1010   (+122)                  1111 0110   (−10)
+ 0001 1101   (+29)                 + 0000 1010   (+10)
  ---------                           ---------
  1001 0111   (shows −105)            1 0000 0000   (0)
carry into MSB = 1, carry out = 0     carry into MSB = 1, carry out = 1
different → OVERFLOW (151 > 127)      same → no overflow, result 0 ✔
```

Binary codes

REFERENCE TABLE

Code	Idea	Example
BCD (8421)	each decimal digit → 4 bits; 1010–1111 invalid	47 → 0100 0111
Excess-3	BCD + 3 (self-complementing)	5 → 1000
Gray	adjacent values differ in **exactly one bit**	binary 1011 → gray 1110
ASCII	7-bit character code	'A' = 65 = 1000001

NOTES

Binary → Gray: $G_{n-1} = B_{n-1}$, $G_i = B_{i+1} \oplus B_i$ (`1011 → 1, 1⊕0=1, 0⊕1=1, 1⊕1=0 → 1110`). **Gray → binary:** $B_{n-1} = G_{n-1}$, $B_i = B_{i+1} \oplus G_i$. **BCD addition:** if a digit sum exceeds 9, add 6 (0110) to that digit.

Overflow in signed addition

NOTES

Overflow happens when the true sum is outside the representable range. Detect it as: **carry into the MSB ≠ carry out of the MSB**, or equivalently both operands have the same sign but the result has the opposite sign. Adding numbers of **opposite sign never overflows**. Carry-out alone is the overflow flag for **unsigned** arithmetic.

IEEE 754 floating point

REFERENCE TABLE

Format	Sign	Exponent	Mantissa (fraction)	Bias
Single (32-bit)	1	8	23	127
Double (64-bit)	1	11	52	1023

NOTES

Normalised value = $(-1)^s \times 1.m \times 2^{e-bias}$ (a hidden leading 1). Exponent field all 0s → zero / denormal; all 1s → ∞ or NaN.

WORKED EXAMPLE

Worked numerical — encode 13.625 (single precision)

```
13.625 = 1101.101₂ = 1.101101 × 2³
sign = 0
exponent = 3 + 127 = 130 = 1000 0010
mantissa = 1011 0100 0000 0000 0000 000   (drop the leading 1)

bits: 0 10000010 10110100000000000000000  = 0100 0001 0101 1010 0000 … = 0x415A0000
```

COMMON TRAP

Pitfall

Floating-point addition needs the exponents **aligned first**; because mantissas have finite length, `(a + b) + c` may differ from `a + (b + c)` — floating-point addition is **not associative**.

More on IEEE 754 — limits, denormals, rounding

REFERENCE TABLE

Quantity (single precision)	Value
Significand bits (with hidden 1)	24
Largest finite number	$(2 - 2^{-23}) \times 2^{127} \approx 3.4028 \times 10^{38}$ (exponent field 254)
Smallest positive normalised	$2^{-126} \approx 1.1755 \times 10^{-38}$ (exponent field 1)
Smallest positive denormal	$2^{-149} \approx 1.4 \times 10^{-45}$ (exponent field 0, no hidden 1)
Machine epsilon (gap above 1.0)	$2^{-23} \approx 1.19 \times 10^{-7}$ (about 7 decimal digits)
Floats between 1 and 2	2^{23} (evenly spaced)

RULES & FACTS

- **Denormals** (exponent field 0, fraction ≠ 0): value = $(-1)^S \times 0.m \times 2^{-126}$. They fill the gap between 0 and the smallest normalised number (gradual underflow).
- Exponent field 255: fraction 0 → ±∞; fraction ≠ 0 → NaN. Exponent and fraction both 0 → ±0.
- Default rounding: **round to nearest, ties to even**. Spacing between floats doubles each time the exponent increases by 1, so large numbers lose absolute precision.

WORKED EXAMPLE

Worked numerical — adding floats

Q: Add 1.5 and 0.25 in binary floating point.

A: $1.5 = 1.1 \times 2^0$ and $0.25 = 1.0 \times 2^{-2}$. Align the smaller exponent to the larger one by shifting the mantissa right 2 places, add, then renormalise:

```
  1.100  × 2⁰
+ 0.010  × 2⁰      (1.0 × 2⁻² shifted right by 2)
= 1.110  × 2⁰  = 1.75      (already normalised; stored as 0x3FE00000 in double precision)
```

WORKED EXAMPLE

Worked numerical — precision loss

Single precision has 24 significant bits, so integers above $2^{24} = 16\,777\,216$ are no longer all representable: `16777216 + 1` rounds back to **16777216**. In double precision `0.1 + 0.2 = 0.30000000000000004 ≠ 0.3`, because neither 0.1 nor 0.2 has a finite binary expansion. Never test floating-point results with `==`.

Fixed point and range summary

RULES & FACTS

- **Fixed point**: a fixed number of integer and fraction bits. An unsigned Qm.n format with m integer bits and n fraction bits has resolution 2^{-n} and range 0 … $2^m - 2^{-n}$.
- Multiplying two n-bit unsigned numbers needs up to **2n bits** for the product; adding two n-bit numbers needs up to **n + 1** bits.
- Sign extension: to widen a 2's complement number, copy the sign bit into the new high bits (e.g. 1101 → 1111 1101 keeps −3).

2 Boolean Algebra & Logic Gates

Basic gates

REFERENCE TABLE

Gate	Expression	Output is 1 when…
AND	A·B	all inputs are 1
OR	A + B	at least one input is 1
NOT	A'	input is 0
NAND	(A·B)'	at least one input is 0
NOR	(A + B)'	all inputs are 0
XOR	A ⊕ B = A'B + AB'	inputs differ (odd number of 1s)
XNOR	(A ⊕ B)'	inputs are equal

Laws you must know cold

KEY POINT

Identities

```
A + 0 = A      A · 1 = A
A + 1 = 1      A · 0 = 0
A + A = A      A · A = A
A + A' = 1     A · A' = 0
(A')' = A
```

KEY POINT

Absorption / redundancy

```
A + AB = A
A(A + B) = A
A + A'B = A + B
A(A' + B) = AB
```

KEY POINT

De Morgan

```
(A + B)' = A'·B'
(A · B)' = A' + B'
(general): complement of f =
 swap + ↔ ·  and complement every literal
```

KEY POINT

Consensus

```
AB + A'C + BC = AB + A'C
(A+B)(A'+C)(B+C) = (A+B)(A'+C)
```

The consensus term BC is redundant — but it is exactly the term that **removes hazards** (Section 4).

NOTES

Duality: swap AND↔OR and 0↔1 (literals unchanged) to get the dual of an identity — the dual of a true identity is also true. The dual of `A + BC'` is `A·(B + C')`. A function with $f^d = f$ is **self-dual**.

XOR properties

FORMULA / WORKING

```
A ⊕ 0 = A     A ⊕ 1 = A'     A ⊕ A = 0     A ⊕ A' = 1
Commutative and associative.   A ⊕ B ⊕ C = 1  iff an ODD number of inputs is 1  (parity)
The complement of an n-input XOR chain is the even-parity function (1 iff an even number of inputs is 1).
```

Canonical forms & counting

- **Minterm**: product containing every variable once (true or complemented); n variables → 2^n minterms. f = Σm(…) (sum of minterms, canonical SOP).
- **Maxterm**: sum containing every variable once. f = ΠM(…) lists the rows where f = **0**. The minterm numbers of f and the maxterm numbers of f are **complementary sets**.
- Number of distinct Boolean functions of n variables = $\mathbf{2^{2^n}}$ (n = 2: 16, n = 3: 256).
- Number of **self-dual** functions of n variables = $2^{2^{n-1}}$ (n = 3: **16**).

Worked numerical — simplify with consensus

Q: Simplify (A + B)(A' + C)(B + C).

A: This is the dual of the consensus pattern: the term (B + C) is redundant, so the result is (A + B)(A' + C). Expanding: AC + A'B (the BC cross term is absorbed).

Universal gates

NAND alone (and NOR alone) can build every other gate, so each is **functionally complete**. {AND, NOT} and {OR, NOT} are also complete; **{AND, OR} is not** (no way to invert).

```
NOT :  A' = (A·A)'                    (tie the inputs together)
AND :  A·B = ((A·B)')'                (NAND followed by NAND-as-inverter)
OR  :  A + B = (A'·B')'               (NAND of the inverted inputs)
XOR :  4 NAND gates:  N1 = (AB)'; N2 = (A·N1)'; N3 = (B·N1)'; A⊕B = (N2·N3)'
```

Bubble pushing

Any AND-OR (SOP) network can be redrawn as **all-NAND two-level logic** directly: NAND–NAND = AND–OR. Similarly an OR-AND (POS) network becomes NOR–NOR.

3 Minimisation: K-Maps & Quine–McCluskey

K-map rules

RULES & FACTS

- Rows and columns are labelled in **Gray-code order** (00, 01, 11, 10) so neighbouring cells differ in one variable.
- Group 1s (for SOP) in rectangles of size **1, 2, 4, 8, 16** — always a power of 2. A group of 2^k cells removes **k** variables.
- Groups may **wrap around** edges (the map is a torus), and may overlap. Use the **fewest, largest** groups that cover every 1.
- For **POS**, group the 0s to get f' as SOP, then complement (or read the groups as sum terms with literals inverted).
- Every cell in an n-variable map has exactly **n neighbours** (4 in a 4-variable map).

REFERENCE TABLE

Term	Meaning
Implicant	any product term that implies f (a group of 1s)
Prime implicant (PI)	an implicant that cannot be enlarged — a **maximal** group
Essential PI (EPI)	a PI covering at least one 1 that **no other PI** covers — it must be in every minimal cover
Don't care (d / X)	input combination that never occurs or whose output is irrelevant; treat as 1 only if it enlarges a group, never need to cover it

WORKED EXAMPLE

Worked numerical — 4-variable map

Q: f(A,B,C,D) = Σm(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15). Find all prime implicants, the essential ones, and a minimal SOP.

AB \ CD	00	01	11	10
00	1_0	0_1	1_3	1_2
01	0_4	1_5	1_7	0_6
11	0_{12}	1_{13}	1_{15}	0_{14}
10	1_8	1_9	1_{11}	1_{10}

```
Prime implicants (6):  B'D' {0,2,8,10}   BD {5,7,13,15}   CD {3,7,11,15}
                       B'C {2,3,10,11}    AD {9,11,13,15}   AB' {8,9,10,11}
Essential (2):  B'D'  (only PI covering m0)   and   BD (only PI covering m5)
Left over after the EPIs: m3, m9, m11
   → pick one PI for m3/m11 (CD or B'C) and one for m9 (AD or AB')

Minimal SOP:  f = B'D' + BD + CD + AD        → 4 terms, 8 literals
(several equal-cost minimal forms exist — e.g. replacing CD by B'C)
```

WORKED EXAMPLE

Worked numerical — a map with NO essential prime implicants

Q: f(A,B,C) = Σm(0, 1, 2, 5, 6, 7).

A \ BC	00	01	11	10
0	1_0	1_1	0_3	1_2
1	0_4	1_5	1_7	1_6

```
Every 1 is covered by TWO pairs → six PIs, none essential (a "cyclic" map):
  A'B' {0,1}  A'C' {0,2}  BC' {2,6}  AB {6,7}  AC {5,7}  B'C {1,5}
Minimal covers use 3 pairs, e.g.  f = A'C' + AB + B'C   or
f = A'B' + BC' + AC   (3 terms, 6 literals)
```

WORKED EXAMPLE

Worked numerical — don't cares

Q: f(A,B,C,D) = Σm(1, 4, 6, 7, 8, 9, 10, 11, 15) + d(0, 5). Find the minimal SOP.

AB \ CD	00	01	11	10
00	X_0	1_1	0_3	0_2
01	1_4	X_5	1_7	1_6
11	0_{12}	0_{13}	1_{15}	0_{14}
10	1_8	1_9	1_{11}	1_{10}

```
Prime implicants (6):  B'C' {0,1,8,9}   A'C' {0,1,4,5}   A'B {4,5,6,7}
                       AB' {8,9,10,11}   BCD {7,15}   ACD {11,15}
Essential: A'B (only PI covering m6) and AB' (only PI covering m10).
Remaining: m1 → B'C' (or A'C'),  m15 → BCD (or ACD)

Minimal SOP:  f = A'B + AB' + B'C' + BCD      → 4 terms, 9 literals
(here A'B + AB' is the XOR of A and B)
```

The don't cares d0 and d5 were used as 1s to enlarge groups; they did **not** have to be covered.

POS from a map

WORKED EXAMPLE

Worked numerical

For the first function above, the 0s are m1, 4, 6, 12, 14. Grouping them: `BD'` {4,6,12,14} and `A'B'C'D` {1}, so `f' = BD' + A'B'C'D`. Complementing with De Morgan:

```
f = (B' + D)(A + B + C + D')
```

Quine–McCluskey (tabular) method

RULES & FACTS

1. List all minterms (and don't cares) in binary, grouped by the **number of 1s**.
2. Repeatedly combine pairs from **adjacent groups** that differ in exactly one bit, replacing that bit by **–**; mark combined terms as used.
3. Terms that can no longer combine are the **prime implicants**.
4. Build a PI chart (rows = PIs, columns = minterms, **excluding don't cares**); pick essential PIs first, then cover the rest with the fewest/cheapest PIs (Petrick's method for cyclic charts).

GATE FAVOURITE

GATE angle

Typical questions: "number of prime implicants", "number of essential prime implicants", "minimum number of literals/gates/NAND gates". Remember: minimum SOP ≠ unique; PIs count is unique; and the answer changes if you must also realise the complement or use POS.

4 Combinational Circuits

Adders and subtractors

REFERENCE TABLE

Circuit	Outputs
Half adder	Sum = A ⊕ B, Carry = A·B
Full adder	Sum = A ⊕ B ⊕ Cin, Cout = AB + Cin(A ⊕ B) = AB + BCin + ACin (majority)
Half subtractor	Diff = A ⊕ B, Borrow = A'B
Full subtractor	Diff = A ⊕ B ⊕ Bin, Bout = A'B + (A ⊕ B)'Bin

NOTES

A **full adder = 2 half adders + 1 OR gate**. An **n-bit adder/subtractor** uses n full adders, with each B input XORed with a control line M and Cin = M: M = 0 adds, M = 1 computes A + B' + 1 = A − B.

Ripple-carry vs carry-lookahead

RULES & FACTS

- **Ripple-carry**: each stage waits for the previous carry, so delay grows **linearly** with n.
- **Carry-lookahead (CLA)**: define **generate** $G_i = A_iB_i$ and **propagate** $P_i = A_i \oplus B_i$. Then $C_{i+1} = G_i + P_iC_i$, and unrolling gives every carry directly from the inputs and C_0 in two gate levels:

```
C1 = G0 + P0·C0
C2 = G1 + P1·G0 + P1·P0·C0
C3 = G2 + P2·G1 + P2·P1·G0 + P2·P1·P0·C0
Sum_i = P_i ⊕ C_i
```

WORKED EXAMPLE

Worked numerical — 4-bit CLA

Q: A = 1011, B = 0110, C0 = 0. Find P, G, all carries and the sum.
A: (bit 0 is the rightmost)

```
bit :   3 2 1 0
A   :   1 0 1 1
B   :   0 1 1 0
P=A⊕B:  1 1 0 1      G=A·B:  0 0 1 0
C1 = G0 + P0C0 = 0 + 1·0 = 0
C2 = G1 + P1C1 = 1 + 0·0 = 1
C3 = G2 + P2C2 = 0 + 1·1 = 1
C4 = G3 + P3C3 = 0 + 1·1 = 1
Sum_i = P_i ⊕ C_i :  S0 = 1⊕0 = 1,  S1 = 0⊕0 = 0,  S2 = 1⊕1
= 0,  S3 = 1⊕1 = 0
Result = C4 S3 S2 S1 S0 = 1 0001 = 17 = 11 + 6 ✔
```

Multiplexer (MUX), demultiplexer, decoder, encoder

REFERENCE TABLE

Block	Inputs → Outputs	Key fact
2^n:1 MUX	2^n data + n select → 1	output = Σ (minterm of selects · data input); a 2^n:1 mux needs $\mathbf{2^n - 1}$ 2:1 muxes
1:2^n DEMUX	1 data + n select → 2^n	a decoder with enable used as data input
n-to-2^n decoder	n → 2^n (one-hot)	generates all minterms; any function = OR of its minterm outputs
2^n-to-n encoder	one-hot → n-bit code	assumes exactly one input active; a **priority encoder** resolves several active inputs
Comparator	A, B → A>B, A=B, A<B	bit equality is XNOR; A = B iff all bit-XNORs are 1

WORKED EXAMPLE

Worked numerical — function with a 4:1 MUX

Q: Implement f(A,B,C) = Σm(1,2,4,7) using a 4:1 MUX with A and B as select lines.
A: For each value of AB, look at f for C = 0 and C = 1:

```
AB = 00  → m0, m1 : f = 0, 1  → f = C       I0 = C
AB = 01  → m2, m3 : f = 1, 0  → f = C'      I1 = C'
AB = 10  → m4, m5 : f = 1, 0  → f = C'      I2 = C'
AB = 11  → m6, m7 : f = 0, 1  → f = C       I3 = C
```

So I0 = C, I1 = C', I2 = C', I3 = C. In general an **n-variable** function needs only a $\mathbf{2^{n-1}}$**:1 MUX** plus at most one inverter. (This f is the 3-input XOR.)

WORKED EXAMPLE

Worked numerical — building bigger decoders

Q: How many 2-to-4 decoders (with enable) build a 4-to-16 decoder? And how many 3-to-8 decoders plus one 2-to-4 for a 5-to-32?
A: 16 outputs / 4 = 4 decoders in the second level, plus 1 decoder in the first level to enable one of them → **5**. For 5-to-32: 32 / 8 = 4 decoders (3-to-8) at the second level + 1 (2-to-4) at the first → **5 decoders** in total.

Hazards (glitches)

NOTES

A **hazard** is a momentary wrong output caused by unequal gate delays when inputs change.

RULES & FACTS

- **Static-1 hazard**: output should stay 1 but briefly drops to 0 — occurs in SOP when two adjacent 1-cells are covered by **different** product terms with no common term.
- **Static-0 hazard**: output should stay 0 but pulses 1 (in POS circuits).
- **Dynamic hazard**: output changes 3 or more times when it should change once.

WORKED EXAMPLE

Worked numerical — removing a static-1 hazard

`f = AB + A'C`. With B = C = 1 and A switching 1 → 0, f should stay 1, but AB falls before A'C rises (the inverter delay) → glitch. Add the **consensus term** BC, which stays 1 throughout: `f = AB + A'C + BC`. On the K-map this is the extra group straddling the two adjacent cells.

COMMON TRAP

Remember

Hazard-free covers need **every pair of adjacent 1s covered by some single product term** — that can mean using non-essential PIs, so the hazard-free circuit is not the minimal one.

5 Latches & Flip-Flops

SR latch

NOTES

Two cross-coupled NOR (or NAND) gates. For the NOR version: S = 1 sets Q = 1, R = 1 resets Q = 0, S = R = 0 holds, and **S = R = 1 is forbidden** (both outputs go 0, and the next state is unpredictable if both return to 0 together). The NAND version is active-low (S' and R'), with S' = R' = 0 forbidden. Adding a clock/enable gives a **gated SR latch**; connecting S = D, R = D' removes the forbidden state and gives the **D latch**.

Characteristic equations and tables

REFERENCE TABLE

Type	Characteristic equation	Behaviour
SR	$Q^+ = S + R'Q$ (SR = 0)	00 hold, 01 reset, 10 set, 11 invalid
D	$Q^+ = D$	copies D on the clock edge
JK	$Q^+ = JQ' + K'Q$	00 hold, 01 reset, 10 set, 11 **toggle**
T	$Q^+ = T \oplus Q = TQ' + T'Q$	0 hold, 1 toggle

NOTES

Excitation tables (what inputs are needed for a wanted Q → Q^+ transition; X = don't care):

REFERENCE TABLE

Q → Q^+	S	R	J	K	D	T
0 → 0	0	X	0	X	0	0
0 → 1	1	0	1	X	1	1
1 → 0	0	1	X	1	0	1
1 → 1	X	0	X	0	1	0

COMMON TRAP

Memory hook

JK with J = K = 1 **toggles**, so it acts as a T flip-flop; JK with K = J' acts as a D flip-flop. JK is the most flexible, T is the building block of counters, D is the building block of registers.

Flip-flop conversions

REFERENCE TABLE

Build…	From…	Connection
T	JK	J = K = T
D	JK	J = D, K = D'
D	SR	S = D, R = D'
T	D	D = T ⊕ Q
JK	D	D = JQ' + K'Q

Triggering and the race-around condition

RULES & FACTS

- **Level-triggered** (latch): output follows input for the whole time the clock is high.
- **Edge-triggered**: samples inputs only at the rising (or falling) edge.
- **Race-around**: a level-triggered JK with **J = K = 1** toggles repeatedly while the clock stays high, if the clock pulse is **longer than the propagation delay**. Cures: **master–slave** or **edge-triggered** design, or a pulse shorter than t_{pd} (impractical).
- **Master–slave**: two latches on opposite clock phases — the master samples while CLK = 1, the slave copies at the falling edge, so the output changes once per cycle.

Timing parameters

RULES & FACTS

- **Setup time** t_{su}: data must be stable **before** the active edge. **Hold time** t_h: data must stay stable **after** the edge. **Clock-to-Q** t_{cq}: delay from the edge to the new output.
- Minimum clock period between two flip-flops: $T \geq t_{cq} + t_{comb(max)} + t_{su}$ (plus clock skew). Hold condition: $t_{cq(min)} + t_{comb(min)} \geq t_h$. Violating setup is cured by a slower clock; violating hold is **not**.

WORKED EXAMPLE

Worked numerical — maximum clock frequency

Q: t_{cq} = 2 ns, longest combinational delay = 5 ns, t_{su} = 1 ns (no skew). Find the maximum clock frequency.
A: T_{min} = 2 + 5 + 1 = 8 ns → f_{max} = 1 / 8 ns = **125 MHz**.

6 Counters & Shift Registers

Asynchronous (ripple) counters

NOTES

Each flip-flop (toggle mode, J = K = 1 or T = 1) is clocked by the **output of the previous one**, so the clock ripples through. An n-bit ripple counter counts 0 … $2^n - 1$ (modulus 2^n); each stage **divides frequency by 2**, so the n-th output has frequency $f/2^n$.

RULES & FACTS

- **Up counter**: clock from Q (falling-edge FFs) or from Q' (rising-edge FFs). **Down counter**: the opposite connection.
- **Drawback**: delays add up. For an n-bit ripple counter the output settles after **n × t_{pd}**, so $f_{max} \approx 1 / (n \cdot t_{pd})$ and decoding glitches appear while it ripples.

WORKED EXAMPLE

Worked numerical

Q: A 4-bit ripple counter uses flip-flops with t_{pd} = 20 ns. Maximum clock frequency?
A: Settling time = 4 × 20 = 80 ns → f_{max} = 1 / 80 ns = **12.5 MHz**. A synchronous 4-bit counter needs only one t_{pd} plus the AND-gate delay, so it runs much faster.

Synchronous counters

NOTES

All flip-flops share **one clock**, so every output changes together after a single t_{pd}. The T-inputs come from combinational logic; for an up counter $T_i = Q_0 \cdot Q_1 \cdot \ldots \cdot Q_{i-1}$. They are faster but need extra gates.

Modulo-N counters

RULES & FACTS

- Flip-flops needed: $\lceil \log_2 N \rceil$ (mod-10 → 4, mod-60 → 6).
- With an **asynchronous reset**: decode state N with a NAND gate and feed it to the clear inputs. The invalid state N exists only for a few nanoseconds (a glitch).
- Output frequency of a mod-N counter = f_{clk} / N (so 1.6 MHz into a mod-16 gives 100 kHz at its MSB).

WORKED EXAMPLE

Worked numerical — mod-6 from a 3-bit ripple counter

```
Count: 000 → 001 → 010 → 011 → 100 → 101 → (110 appears for an instant) → 000
```

Decode **Q2 Q1 Q0 = 110** (the first state ≥ 6; Q2 and Q1 are enough to detect it) with a NAND and drive all CLEAR lines. Counts 0–5 are stable, so the modulus is 6.

Shift registers

REFERENCE TABLE

Type	Load	Read	Clocks for n bits
SISO	serial	serial	n to load, n more to read out
SIPO	serial	parallel	n to load, then read at once
PISO	parallel	serial	1 to load, n to read out
PIPO	parallel	parallel	1

NOTES

A serial-to-parallel conversion of an n-bit word needs **n clock pulses**. Shifting left by one position doubles an unsigned number.

Ring and Johnson (twisted-ring) counters

KEY POINT

Ring counter
Q of the last FF fed to D of the first. One circulating 1.
n flip-flops → n states

```
1000 → 0100 → 0010 → 0001 → 1000
```

Needs initial preset; self-starting only with extra logic.

KEY POINT

Johnson counter
Q' of the last FF fed to the first.
n flip-flops → 2n states

```
0000 → 1000 → 1100 → 1110 → 1111
→ 0111 → 0011 → 0001 → 0000
```

Adjacent states differ in **one bit**, so decoding is glitch-free; each state decodes with a 2-input AND.

GATE FAVOURITE

GATE angle

Mod comparison: n flip-flops give 2^n states as a binary counter, 2n states as a Johnson counter, and only n states as a ring counter. The unused states of Johnson/ring counters form **lock-out** cycles unless corrected.

7 Finite State Machines & Synchronous Design

Mealy vs Moore

REFERENCE TABLE

	Moore	Mealy
Output depends on	**present state only**	**present state AND present input**
Output labelled on	states	transitions (input / output)
Typical state count	more states	fewer states
Output timing	changes after clock edge (synchronous, glitch-free)	can change as soon as input changes (reacts a cycle earlier, may glitch)

RULES & FACTS

- Moore → Mealy: always possible with the **same number of states**.
- Mealy → Moore: may need up to **(states × distinct outputs)** states.
- Flip-flops needed for N states: $\lceil \log_2 N \rceil$ (5 states → 3 flip-flops).

FORMULA / WORKING

```
Mealy machine  (next state / output)
State    input 0     input 1
S0       S0 / 0      S1 / 0
S1       S2 / 0      S1 / 0
S2       S0 / 0      S3 / 0
S3       S2 / 0      S1 / 1      ← the final 1 of 1011 completes the pattern; suffix "1" stays → S1
```

WORKED EXAMPLE

Worked numerical — states, flip-flops and a trace

Mealy machine: **4 states** → **2 flip-flops**. Moore machine: needs an extra state S4 ("1011", output 1) → **5 states** → **3 flip-flops**.

```
input :  1 0 1 1 0 1 1 0 1 0 1 1 1
output:  0 0 0 1 0 0 1 0 0 0 0 1 0        → three detections
(overlaps at the 4th and 7th bits count)
```

WORKED EXAMPLE

Worked numerical — 101 detector, overlapping

Three Mealy states (none / "1" / "10"); on state "10" with input 1 the output is 1 and the next state is "1". For input `1101011` the output is `0001010`.

State minimisation

NOTES

Two states are **equivalent** if, for every input sequence, they produce the same outputs (and move to equivalent states). Procedure: partition states by output behaviour, then keep splitting while their next states fall in different blocks (partition refinement — the same idea as DFA minimisation). Equivalent states merge, reducing the number of flip-flops and logic.

GATE FAVOURITE

GATE angle

"How many states / flip-flops for this detector?" and "what is the output sequence for this input?" are the two commonest questions. Always check whether the machine is **overlapping or non-overlapping**, and Mealy or Moore, before counting.

Synchronous design procedure

RULES & FACTS

1. Draw the state diagram / state table from the specification.
2. **Minimise states** (merge equivalent states).
3. **Assign binary codes** to states (number of flip-flops = $\lceil \log_2 N \rceil$).
4. Use the flip-flop **excitation table** to derive flip-flop input equations and output equations (K-maps; unused states are don't cares).
5. Draw the circuit; check that unused states eventually return to a valid state (self-starting).

Sequence detector: overlapping 1011

NOTES

State names track the longest suffix of the input that is a prefix of 1011: S0 = nothing, S1 = "1", S2 = "10", S3 = "101".

Worked design — a synchronous mod-5 counter with T flip-flops

NOTES

Design a counter that follows `000 → 001 → 010 → 011 → 100 → 000`. The states 101, 110, 111 never occur, so they are **don't cares**. For a T flip-flop the excitation is $T = Q \oplus Q^+$ (T = 1 exactly when that bit must change).

FORMULA / WORKING

Present Q2Q1Q0	Next Q2Q1Q0	T2 T1 T0
000	001	0 0 1
001	010	0 1 1
010	011	0 0 1
011	100	1 1 1
100	000	1 0 0
101,110,111	X X X	X X X

NOTES

K-map each T over the 3 present-state variables, using the don't cares to enlarge groups:

FORMULA / WORKING

```
T0 = Σm(0,1,2,3) + d(5,6,7)   →  T0 = Q2'
T1 = Σm(1,3)     + d(5,6,7)   →  T1 = Q0
T2 = Σm(3,4)     + d(5,6,7)   →  T2 = Q2 + Q1·Q0
```

NOTE

Self-start check (always do this)

Feed the unused states through the final equations:

```
101 → T2T1T0 = 1 1 0 → 011        110 → T = 1 0 0 → 010
111 → T = 1 1 0 → 001
```

Every unused state falls into the main cycle within one clock, so the counter is **self-starting**. Counter modulus = 5 → $\lceil \log_2 5 \rceil$ = 3 flip-flops.

8 Memory & Programmable Logic

Memory sizing

RULES & FACTS

- A memory with **k address lines** holds 2^k words; with w bits per word, capacity = $2^k \times w$ bits. Address lines needed for N words = $\lceil \log_2 N \rceil$.
- 12 address lines × 8-bit words → 4096 × 8 = 32 768 bits = **4 KB**.
- Building a bigger memory from smaller chips: chips needed = (total words ÷ words per chip) × (total width ÷ chip width). A 16K × 16 memory from 4K × 8 chips: (16K/4K) × (16/8) = 4 × 2 = **8 chips**. The extra high-order address lines (2 here) drive a **decoder** that selects the chip row.
- **RAM** is volatile read/write (SRAM: flip-flop cells, fast; DRAM: capacitor cells, dense, needs refresh). **ROM** is non-volatile (mask ROM, PROM, EPROM, EEPROM, Flash).

ROM as a combinational circuit

NOTES

A ROM with n address lines and m data lines is an **n-to-2^n decoder followed by m OR gates**: the decoder produces every minterm and each output is the OR of the minterms stored as 1. Hence a $2^n \times m$ ROM can implement **any m Boolean functions of n variables**; its size is $2^n \times m$ bits. A ROM with 4 inputs and 3 outputs holds 16 × 3 = **48 bits**. It is simple but wasteful when only a few minterms are needed, since the table doubles with each extra input.

PLA, PAL, CPLD, FPGA

REFERENCE TABLE

Device	AND array	OR array	Note
ROM	fixed (full decoder, all minterms)	programmable	every minterm available
PLA	**programmable**	**programmable**	only the needed product terms are generated and can be shared between outputs
PAL	**programmable**	fixed	cheaper and faster, but each output has a limited number of product terms
CPLD	several PAL-like blocks with programmable interconnect		non-volatile, fast fixed timing
FPGA	array of **look-up tables (LUTs)**, flip-flops and routing		a k-input LUT is a $2^k \times 1$ memory and can realise **any** function of k variables (4-input LUT = 16 bits)

WORKED EXAMPLE

Worked numerical — size of a PLA vs a ROM

Q: A circuit has 5 inputs and 4 outputs. Compare a ROM with a PLA that needs 20 product terms.

A: Assume each input is available true and complemented, so the AND array has 2 × 5 = 10 input lines.

```
ROM : 2⁵ × 4                  = 128 programmable bits
PLA : AND array 10 × 20       = 200 links
      OR  array 20 × 4        =  80 links        total
280 links
```

For this function the ROM is smaller. A PLA wins when the number of product terms is far below 2^n (for example 5 inputs but only 6 product terms: 60 + 24 = 84 links).

Other useful building blocks

RULES & FACTS

- **Tri-state buffer**: output is 0, 1 or high-impedance (Z). Several tri-state drivers can share one **bus** if exactly one is enabled at a time.
- **Open-collector / open-drain** outputs allow a **wired-AND** on a shared line with a pull-up resistor.
- **Fan-in** = number of inputs a gate accepts; **fan-out** = number of gate inputs one output can drive. A wide AND with limited fan-in is built as a tree of smaller gates, adding delay of about $\lceil \log_k n \rceil$ levels.
- **Propagation delay**: critical path = the longest delay from any input to any output; it sets the clock-period limit of a synchronous design.

9 Arithmetic Circuits & Error-Control Codes

BCD adder

NOTES

Add two BCD digits with a 4-bit binary adder. If the binary sum is greater than 9 (that is, the carry-out is 1, or the sum is 1010–1111), **add 6 (0110)** to correct it and propagate a decimal carry. The correction condition is $K = C_{out} + S_3S_2 + S_3S_1$.

WORKED EXAMPLE

Worked numerical

8 + 5 = 13: 1000 + 0101 = 1101 (13 > 9) → add 0110: 1101 + 0110 = 1 0011 → carry 1, digit 0011 → BCD **0001 0011** = 13 ✔.

Multipliers

RULES & FACTS

- **Shift-and-add**: for each multiplier bit that is 1, add the shifted multiplicand — n cycles for n-bit operands, 2n-bit product.
- **Array multiplier** (n × n): n^2 AND gates generate the partial products; they are added by n(n−1) adders in total (n(n−2) full adders and n half adders). A 4 × 4 array multiplier therefore uses **16 AND gates, 8 full adders and 4 half adders**. It is fast (delay grows linearly with n) but large.
- **Booth's algorithm** handles signed 2's complement operands by recoding runs of 1s in the multiplier (the pair Q_0Q_{-1} = 10 → subtract, 01 → add, 00/11 → shift only).
- Adders recap: ripple-carry delay ∝ n; carry-lookahead is about constant to log n; **carry-select** and **carry-skip** adders sit in between.

Parity and Hamming distance

RULES & FACTS

- **Even parity**: add a bit so the total number of 1s is even (odd parity: odd). A single parity bit detects any **odd** number of bit errors but cannot locate them and misses even-numbered errors.
- **Hamming distance** d(x, y) = number of positions where x and y differ. The minimum distance d_{min} of a code decides its power:

```
detect up to  d_min − 1  errors
correct up to ⌊(d_min − 1)/2⌋ errors
detect e and correct t simultaneously if d_min ≥ e + t + 1  (e ≥ t)
```

- A code with d_{min} = 5 detects up to 4 errors or corrects up to 2. A code with d_{min} = 3 corrects 1 error; adding an overall parity bit makes d_{min} = 4 (**SECDED**: single-error-correct, double-error-detect).

WORKED EXAMPLE

Worked numerical

d(10110, 11011): XOR = 01101 → three 1s → distance **3**. Even parity bit for 1011011 (five 1s) = **1**.

Hamming (7,4) single-error-correcting code

NOTES

For m data bits you need r check bits with $2^r \geq m + r + 1$: m = 4 → r = 3 (7-bit code word), m = 8 → r = 4 (12-bit word). Check bits sit at the **power-of-two positions** 1, 2, 4, …; check bit p_i covers every position whose binary index contains bit i.

FORMULA / WORKING

```
Position:   1   2   3   4   5   6   7
Role:       p1  p2  d1  p4  d2  d3  d4
p1 covers positions 1,3,5,7     p2 covers 2,3,6,7     p4 covers 4,5,6,7  (even parity)
```

WORKED EXAMPLE

Worked numerical — encode and correct

Encode data 1011 (d1 d2 d3 d4 = 1,0,1,1):

```
p1 = d1⊕d2⊕d4 = 1⊕0⊕1 = 0
p2 = d1⊕d3⊕d4 = 1⊕1⊕1 = 1
p4 = d2⊕d3⊕d4 = 0⊕1⊕1 = 0
code word (positions 1…7) = 0 1 1 0 0 1 1  =  0110011
```

Receive 0110111 (bit 5 flipped). Recompute the three checks over their groups; the failed checks form the **syndrome**:

```
c1 (1,3,5,7) = 0⊕1⊕1⊕1 = 1       c2 (2,3,6,7) = 1⊕1⊕1⊕1 = 0
c4 (4,5,6,7) = 0⊕1⊕1⊕1 = 1
syndrome c4c2c1 = 101 = 5  →  the error is in position 5:
flip it back → 0110011 → data bits (3,5,6,7) = 1011
```

The syndrome, read as a binary number, **is the position of the error** (0 means no error). All 16 × 7 single-bit-error cases were checked by program.

CRC (brief)

NOTES

A **cyclic redundancy check** divides the message (shifted left by the degree of the generator polynomial) by the generator using XOR (mod-2) division; the remainder is appended. A receiver recomputes it; a non-zero remainder signals an error. A CRC with a degree-r generator detects all burst errors of length ≤ r and all single-bit errors if the generator has at least two terms.

10 High-Yield Sequential & Boolean Methods

Asynchronous sequential circuits

MUST REMEMBER

Fundamental-mode assumption

For fundamental-mode analysis, only one input changes at a time and the circuit is allowed to reach a stable state before the next input change. Do not apply synchronous edge-triggered assumptions.

GATE FAVOURITE

Stable-state analysis

Write the next-state equation, identify states satisfying $Q_{next}=Q$ for the current input, then check whether transitions under the permitted input changes lead to the intended stable state.

COMMON TRAP

Race vs hazard

A **race** concerns multiple state variables changing and potentially reaching different final states. A **hazard** is an unwanted transient caused by unequal propagation delays in a combinational path.

MUST REMEMBER

Critical race

A race is critical when different possible orders of state-variable changes can lead to different stable states. State assignment should avoid assigning adjacent transitions to codes requiring multiple simultaneous bit changes.

GATE FAVOURITE

Essential hazard

Occurs in asynchronous circuits because a feedback path has unequal delays. Unlike a static/dynamic logic hazard, it cannot generally be removed by merely adding a redundant combinational consensus term; the feedback path must be designed with appropriate delays/structure.

FORMULA / RULE

Static-1 hazard

For SOP logic, if adjacent 1-minterms are covered by separate product terms but not a common implicant, unequal delays can create a temporary 0. Add the consensus/overlap implicant when appropriate.

Timing-diagram workflow

METHOD

How to solve a flip-flop timing question

1. Identify the active clock edge/level.
2. Evaluate J/K, D or T immediately before the active edge.
3. Apply the characteristic equation: D → Q_{next}=D; T → Q_{next}=Q⊕T; JK → 00 hold, 01 reset, 10 set, 11 toggle.
4. Apply preset/clear first if asynchronous controls are asserted.
5. Only then draw Q and account for clock-to-Q delay if the question asks for waveform timing.

State assignment

SHORTCUT

Adjacent transitions

Prefer state codes that differ in one bit for transitions that occur frequently. This reduces race risk in asynchronous implementations and often simplifies excitation equations.

COMMON TRAP

Don't confuse minimisation with assignment

State minimisation merges equivalent states; state assignment maps the remaining states to binary codes. They are different design stages.

LFSR

MUST REMEMBER

Linear feedback

An n-bit maximal-length LFSR can generate at most 2^n-1 nonzero states when its feedback polynomial is primitive. The all-zero state is a lock-up state for a standard XOR-feedback LFSR.

FORMULA / RULE

State transition method

Write the current register, compute the feedback XOR from the specified taps, shift in the feedback bit, and repeat until a state repeats. The number of distinct nonzero states is the period.

Shannon expansion

FORMULA / RULE

Shannon expansion

For variable x: $F = x \cdot F|_{x=1} + x' \cdot F|_{x=0}$. It is the algebraic basis of MUX implementation: use x as the select and the two cofactors as the data inputs.

Representative worked problems

METHOD

Q1 · LFSR period

Problem: An n=4 maximal-length XOR LFSR is initialized to any nonzero state. What is its maximum period?

Method: A primitive 4-bit LFSR visits every nonzero state exactly once.

$2^4-1 = 15$. Maximum period = **15 clocks**. The zero state must be avoided.

METHOD

Q2 · Shannon expansion / MUX

Problem: Express $F(A,B,C)=\Sigma m(1,2,5,7)$ using A as the MUX select.

Method: Compute cofactors. For A=0, minterms 1,2 correspond to BC=01,10, giving $F_0=B \oplus C$. For A=1, minterms 5,7 correspond to BC=01,11, giving $F_1=C$. Therefore $F=A' \cdot (B \oplus C)+A \cdot C$. A 2:1 MUX uses select A, D0=B⊕C and D1=C.

METHOD

Q3 · Static-1 hazard recognition

Problem: A two-level SOP changes from one input combination where F=1 to an adjacent combination where F=1, but the two points are covered by different product terms with no common implicant. What failure should you suspect?

Method: Adjacent 1-cells should have an overlapping implicant for a hazard-free static-1 transition.

Suspect a **static-1 hazard**. Add the appropriate consensus/overlap term if it is logically redundant and permitted by the implementation.

METHOD

Q4 · Flip-flop timing

Problem: A positive-edge D flip-flop sees D=1 immediately before one rising edge and D=0 immediately before the next rising edge. Ignore asynchronous inputs. What are the sampled Q values?

Method: Sample D only at the active edge; between edges Q retains its stored value.

After the first rising edge Q becomes **1**; after the second rising edge Q becomes **0**. Changes in D between those edges do not immediately change Q.

11 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- For n variables, a complete truth table has 2^n rows.
- For a k-bit binary counter, the number of states is 2^k.
- **Shannon:** F = x·F|x=1 + x'·F|x=0; the two cofactors can directly feed a 2:1 MUX.

LAST-MINUTE · MUST REMEMBER

- **De Morgan:** (AB)' = A'+B' and (A+B)' = A'B'.
- Use don't-care terms when they are permitted to simplify the implementation; they may be treated as 0 or 1 as convenient.
- MUX can implement Boolean functions; a 2^n:1 MUX can directly realise functions of n select variables plus suitable data inputs.
- Latch is level-sensitive; flip-flop is edge-triggered in the standard synchronous design model.
- Setup and hold time violations can cause metastability; clock-to-Q is a propagation characteristic, not a setup requirement.
- Synchronous counters share a common clock; ripple/asynchronous counters propagate state changes stage by stage.
- Combinational circuits have no memory; sequential circuits depend on present inputs and stored state.
- **Asynchronous circuits:** assume one input change at a time and allow the circuit to settle before the next change.
- A maximal-length n-bit XOR LFSR has period 2^n-1 and must avoid the all-zero lock-up state.

LAST-MINUTE · COMMON TRAPS

- K-map groups must contain 1,2,4,8,... cells and be rectangular; wrapping across edges is allowed.
- Race is a state-transition problem; hazard is a propagation-delay transient.

LAST-MINUTE · GATE FAVOURITES

- For FSMs, distinguish Moore output dependence on state from Mealy output dependence on state and input.